Lesson 7: Error Handling and Information Leakage

Marcin Kurzyna

Lecture Goals

This lecture introduces:

- Why errors are dangerous,
- How attackers use error messages,
- Safe exception handling,
- Information leakage,
- Basic secure logging ideas.

1 Why Errors Matter

Programs often fail because of:

- Invalid input,
- Missing files,
- Wrong passwords,
- Programming mistakes.

A crash is not only a bug:

It can become a security problem.

2 What Is Information Leakage?

Information leakage means exposing data that should stay hidden.

Examples:

- File paths,
- Database details,
- Passwords,
- Internal system information,
- Stack traces.

Attackers use small details to learn about systems.

3 Example: Dangerous Error Message

```
password = input("Password: ")

if password != "secret":
    print("Wrong password: expected secret")
```

Problem:

- Program reveals the correct password.

4 Safer Version

```
password = input("Password: ")

if password != "secret":
    print("Access denied")
```

Good security practice:

> Give attackers as little information as possible.

5 Program Crashes

Example:

```
number = int(input("Enter number: "))
print(100 / number)
```

Possible problems:

- User enters text,
- User enters zero.

Program may crash.

6 Using try/except

```
try:
    number = int(input("Enter number: "))
    print(100 / number)

except:
    print("Invalid input")
```

Now the program:

- Does not crash,
- Handles errors safely.

7 Why Broad Exceptions Are Dangerous

This code hides all errors:

```
try:
    run_program()
except:
    print("Error")
```

Problem:

- Real bugs become invisible,
- Debugging becomes difficult.

8 Better Exception Handling

```
try:
    number = int(input("Enter number: "))
    print(100 / number)

except ValueError:
    print("Please enter a valid number")

except ZeroDivisionError:
    print("Cannot divide by zero")
```

Advantages:

- Safer,
- Clearer,
- Easier to debug.

9 File Errors

Example:

```
file = open("secret.txt")
content = file.read()
print(content)
```

Possible problem:

- File does not exist.

10 Dangerous Debug Output

```
try:
    file = open("secret.txt")
except Exception as e:
    print(e)
```

Possible output:

[Errno 2] No such file or directory: '/home/admin/project/secret.txt'

Problem:

- Attacker learns server structure.

11 Safer Error Messages

```
try:
    file = open("secret.txt")

except FileNotFoundError:
    print("File unavailable")
```

Internal details stay hidden.

12 Debug Mode vs Production

During development:

- Detailed errors are useful.

In production:

- Detailed errors are dangerous.

Never expose:

- Stack traces,
- Passwords,
- API keys,
- Internal paths.

13 Logging

Programs often save logs.

Example:

```
print("User login failed")
```

Logs help:

- Detect attacks,
- Find bugs,
- Monitor systems.

14 Unsafe Logging

```
password = input("Password: ")
print("Password entered:", password)
```

Problem:

- Passwords appear in logs.

Sensitive data should never be logged.

15 Security Principle

> Fail securely.

This means:

- Handle errors safely,
- Avoid crashes,
- Hide sensitive information,
- Keep systems stable.

Summary

In this lecture, we covered:

- Why errors are security risks,
- Information leakage,
- Safe exception handling,
- Secure error messages,
- Basic logging security.

16 Exercises

1. Find problems in the code:

```
password = input("Password: ")

if password != "admin123":
    print("Correct password is admin123")
```

2. Improve this code:

```
number = int(input("Enter number: "))
print(50 / number)
```

3. Why is this dangerous?

```
except Exception as e:
    print(e)
```

4. Write a safer version of a program that opens a file.
5. Give 3 examples of sensitive information that should not appear in logs.
6. Challenge:

   Design a login system that:

   - Handles errors safely,
   - Does not reveal passwords,
   - Does not crash on invalid input.